TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN CƠ SỞ

ÁP DỤNG HỌC SÂU ĐỂ NHẬN DẠNG GIỐNG CHÓ MÈO

Giảng viên hướng dẫn: TRẦN CHÂU THANH THIỆN

Sinh viên thực hiện: NGUYỄN XUÂN TIẾN

MSSV: 2311558529

Khoá: 2023

Ngành/ chuyên ngành: TRÍ TUỆ NHÂN TẠO

TP. HỒ CHÍ MINH – 7/2025

TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN CƠ SỞ

ÁP DỤNG HỌC SÂU ĐỂ NHẬN DẠNG GIỐNG CHÓ MÈO

Giảng viên hướng dẫn: TRẦN CHÂU THANH THIỆN

Sinh viên thực hiện: NGUYỄN XUÂN TIẾN

MSSV: 2311558529

Khoá: 2023

Ngành/ chuyên ngành: TRÍ TUỆ NHÂN TẠO

TP. HỒ CHÍ MINH – 7/2025

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành đến thầy cô khoa công nghệ thông tin đã tận tình giảng dạy và truyền đạt những kiến thức quý báu, tạo nền tảng vững chắc để em có thể thực hiện đề tài này.

Em xin bày tỏ lòng biết ơn sâu sắc đến thầy Thiện vì đã luôn tận tâm chỉ bảo, đưa ra những định hướng khoa học và đóng góp quan trọng trong suốt quá trình nghiên cứu.

BM-ChT-

TRƯỜNG ĐẠI HỌC
NGUYỄN TẤT THÀNH
TRUNG TÂM KHẢO THÍ

KỲ THI KẾT THÚC HỌC PHẦN
HỌC KỲ 3 NĂM HỌC 2024 - 2025

PHIẾU CHẤM THI TIỂU LUẬN/ĐỒ ÁN

Môn thi: Đồ án cơ sở Trí thuệ nhân tạo .....Lớp học phần: 23DTH2D.....

Nhóm sinh viên thực hiện :

1 Nguyễn Xuân Tiến..... Tham gia đóng góp: thực hiện đồ án.....

Ngày thi: 17/09/2025.....Phòng thi: L.602.....

Đề tài đồ án của sinh viên : Áp dụng học sâu để nhận diện giống chó mèo.....

Phần đánh giá của giảng viên (căn cứ trên thang rubrics của môn học):

Tiêu chí (theo CĐR HP)	**Đánh giá của GV**	**Điểm tối đa**	**Điểm đạt được**
Cấu trúc của báo cáo	....		
Nội dung	....		
Các nội dung thành phần	....		
Lập luận	....		
Kết luận	....		
Trình bày	....		
TỔNG ĐIỂM			

Giảng viên chấm thi

(ký, ghi rõ họ tên)

MỤC LỤC

DANH MỤC HÌNH vii
DANH MỤC BẢNG ix
CÁC TỪ VIẾT TẮT x
Chương 1 GIỚI THIỆU 1
1.1 Lời mở đầu 1
1.2 Công cụ hỗ trợ 2
Chương 2 CƠ SỞ LÝ THUYẾT 3
2.1 CNN (Convolution Networks) 3
2.1.1 Lớp tích chập (Convolution layer) 4
2.1.2 Lớp gộp (Pooling layer) 6
2.1.3 Lớp kết nối đầy đủ (Fully Connected Layer) 6
2.1.4 Điều chỉnh siêu tham số 7
2.1.5 Hàm kích hoạt 9
2.2 Vấn đề về mạng nơ-ron rất sâu 11
2.3 Resnet (Residual Network) 12
2.3.1 Khối đồng nhất (The identity block) 13
2.3.2 Khối tích chập (The convolutional block) 13
2.4 Mobilenet_v2 14
2.4.1 Tích chập phân tách theo chiều sâu (depth-wise separable convolution) 15
2.5 Efficientnetv2_s 17
2.6 TRANSFER LEARNING 19
Chương 3 MÔ HÌNH THỰC NGHIỆM 21
3.1 Thu thập và tiền xử lý dữ liệu 21
3.1.1 Tổng quan về dữ liệu 21
3.1.2 Tải và xử lý dữ liệu 22
3.1.2.1 Tải dữ liệu 22
3.1.2.2 Tiền xử lý dữ liệu và nhãn 23
3.1.3 Phân chia và chuẩn hóa dữ liệu 25
3.1.3.1 Phân chia dữ liệu cho huấn luyện mô hình 25
3.1.3.2 Chuẩn hóa dữ liệu 26
3.2 Xây dựng và huấn luyện mô hình 26
3.2.1 Kiến trúc mô hình hoàn chỉnh 27
3.2.2 Cấu Hình và Quá Trình Huấn Luyện 28
3.3 Kết quả 29
Chương 4 KẾT LUẬN VÀ KIẾN NGHỊ 30
4.1 Kết luận 30
4.2 Khiến nghị 30
Tài liệu tham khảo 32

DANH MỤC HÌNH

Hình 2. 1 Phép tính tích chập 4

Hình 2. 2: K bộ lọc được sử dụng cho ra kết quả một bản đồ đặc trưng có kích thước $O \times O \times K$ 5

Hình 2. 3: Phép tính tích chập với bước nhảy là 1 và 2 5

Hình 2. 4: Zero padding 5

Hình 2. 5: Gộp cực đại và gộp trung bình 6

Hình 2. 6: Dữ liệu được làm phẳng khi đưa vào lớp FC 7

Hình 2. 7: Hàm kích hoạt ReLU 10

Hình 2. 8: Hàm kích hoạt Softmax 11

Hình 2. 9: Sự biến mất của gradient khi mạng rất sâu 12

Hình 2. 10: Mối khối ResNet hiện thị "skip connection" 12

Hình 2. 11: Khối đồng nhất với kết nối bỏ qua 2 lớp 13

Hình 2. 12: Khối tích chập trong ResNet 13

Hình 2. 13: Kiến trúc MobileNet_V2 14

Hình 2. 14: Tích chập theo chiều sâu, sử dụng 3 bộ lọc để thay đổi kích thước đầu vào 12x12x3 sang 8x8x3 15

Hình 2. 15: Tích chập điểm, chuyển đổi đầu vào có 3 kênh sang 1 kênh 16

Hình 2. 16: Tích chập điểm với 256 kênh 16

Hình 2. 17: Khối dư đảo ngược (Inverted residual block) 18

Hình 2. 18: Kết quả khi so sánh độ chính xác và các tham số của kiến trúc EfficientNetV2 với một số kiến trúc khác 18

Hình 3. 1: Thành phần dữ liệu Oxford-IIIT. 12 giống mèo và 25 giống chó 21

Hình 3. 2: Một số hình ảnh từ bộ dữ liệu Oxford-IIIT Pet 22

Hình 3. 4: Hiển thị ảnh 5001 trong input_img_paths 23

Hình 3. 5: Lấy nhãn từ đường dẫn 23

Hình 3. 6: Mã hóa nhãn khi huấn luyện (labelEncDict) 24

Hình 3. 7: Giải mã nhãn (labelDecDict) và hiển thị kết quả khi giải mã 24

Hình 3. 8: Chuyển ảnh về mảng numpy và đưa về kích thước 224x224 24

Hình 3. 9: Trực quan hóa vài dữ liệu sau khi đã tiền xử lý 25

Hình 3. 10: Chia dữ liệu với thư viện thư viện scikit-learn 25

Hình 3. 11: Kết quả sau khi chia dữ liệu huấn luyện 26

Hình 3. 12: Chuẩn hóa dữ liệu về khoảng [0, 1] 26

Hình 3. 13: Kiến trúc mô hình hoàn chỉnh 27

Hình 3. 14: Biên dịch mô hình 28

Hình 3. 15: Huấn luyện mô hình với 6 epochs 28

DANH MỤC BẢNG

Bảng 2. 1: So sánh tham số tương thích và không tương tích trong tích chập 8

Bảng 2. 2: Tổng các tham số trong lớp tích chập và đệm 9

Bảng 2.3: Các lớp thuộc kiến trúc mạng EfficientNet-B0 17

Bảng 3.1: Độ chính xác (Acc) trên tập huấn luyện (train) và tập xác thực (val) 29

Bảng 3.2: Độ mất mát (Loss) trên tập huấn luyện (train) và tập xác thực (val) 29

CÁC TỪ VIẾT TẮT

CNN	Convolutional Neural Networks
ResNet	Residual Network
FC	Fully Connected Layer
ReLU	Rectified Linear Unit
Acc	Accuracy
AI	Artificial Intelligence

CHƯƠNG 1

GIỚI THIỆU

1.1 LỜI MỞ ĐẦU

Chó và mèo là hai động vật nuôi phổ biến trên thế giới. Bên cạnh đó, mỗi vùng, mỗi quốc gia thì lại có những giống chó và mèo khác nhau. Sự khác biệt về giống này có thể do nhu cầu của con người cho các mục đích như làm bạn, đi săn, trình diễn quảng cáo,... Chó và mèo có nhiều giống khác nhau và mỗi giống đòi hỏi cách chăm sóc và loại thức ăn riêng biệt. Tuy nhiên, một số ít người nuôi có khả năng nhận biết chính xác giống của thú cưng mình. Điều này dẫn đến việc nhận dạng sai và chăm sóc không đúng cách, gây ra những hậu quả nghiêm trọng.

Các mạng nơ-ron sâu (deep neural networks) như mạng nơ-ron tích chập (CNNs) và bộ biến đổi (transformers) đã đạt được nhiều thành công trong phân loại hình ảnh trong những năm gần đây. Người ta đã chứng minh một cách nhất quán rằng phương pháp tốt nhất cho phân loại hình ảnh là khi các mô hình sâu lớn có thể được huấn luyện trên tập dữ liệu dồi dào và đã được gán nhãn.

Tuy nhiên, có nhiều tình huống trong thế giới thực mà yêu cầu về một lượng lớn dữ liệu huấn luyện để đạt hiệu suất tốt nhất không thể được đáp ứng. Đối với các bài toán chuyên biệt như nhận diện giống chó và mèo, việc thu thập và gán nhãn cho một bộ dữ liệu đủ lớn thường là một thách thức không nhỏ. Để giải quyết những rào cản này, phương pháp học chuyển giao (transfer learning) đã trở thành một giải pháp hiệu quả. Kỹ thuật này cho phép chúng ta tận dụng kiến thức đã được học từ các mô hình được huấn luyện trên những bộ dữ liệu lớn và tổng quát (như ImageNet) để áp dụng cho các tác vụ chuyên biệt hơn. Bằng cách sử dụng các mô hình đã được tiền huấn luyện (pre-trained models) làm nền tảng và chỉ cần tinh chỉnh (fine-tuning) trên bộ dữ liệu nhỏ hơn, chúng ta có thể đạt được hiệu suất cao với chi phí thấp hơn đáng kể về thời gian và tài nguyên.

Nghiên cứu này tập trung vào việc ứng dụng học chuyển giao để xây dựng một hệ thống nhận diện giống chó và mèo. Đánh giá và so sánh hiệu suất của ba kiến trúc mô hình nổi tiếng gồm MobileNetV2, ResNet50 và EfficientNetV2-S trên bộ dữ liệu Oxford-IIIT Pet.

1.2 CÔNG CỤ HỖ TRỢ

- **Ngôn ngữ lập trình Python:** Ngôn ngữ chính để xử lý dữ liệu và xây dựng mô hình.
- **Google colab:** Nơi thực thi Python trên nền tảng đám mây, cung cấp miễn phí CPU Intel Xeon @ 2,2 GHz, VRAM GDDR5 12 GB, GPU 13GB, TPU 180 teraflop.
- **Numpy:** Thư viện cơ bản cho khoa học tính toán, được dùng để xử lý các ma trận và mảng đa chiều.
- **Glob:** Thư viện này dùng để tìm kiếm các tệp theo một mẫu cụ thể. Ta dùng nó để lấy danh sách tất cả các file ảnh (.jpg, .png) trong một thư mục, giúp tự động hóa quá trình xử lý dữ liệu.
- **Matplotlib:** Giúp trực quan hóa dữ liệu.
- **Tensorflow và tensorflow_hub:** Đây là hai thư viện quan trọng trong lĩnh vực học máy và học sâu (Deep Learning). TensorFlow là một framework mã nguồn mở do Google phát triển, cho phép xây dựng và huấn luyện các mô hình học sâu. TensorFlow Hub là một thư viện cung cấp các mô hình học máy được huấn luyện sẵn.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 CNN (CONVOLUTION NETWORKS)

Lần đầu tiên được giới thiệu trong một bài báo nổi tiếng của Yann LeCun và cộng sự vào năm 1989, với phiên bản hoàn chỉnh hơn được công bố vào năm 1998. Mạng Neural tích chập (Convolutional Neural Networks) [1], được thiết kế đặc biệt để xử lý và phân tích các dạng dữ liệu có cấu trúc lưới (grid-like data), tiêu biểu là hình ảnh. Lấy cảm hứng từ cơ chế hoạt động của vỏ não thị giác ở sinh vật, CNN cho phép máy tính "nhìn" và "hiểu" hình ảnh bằng cách tự động học và trích xuất các đặc trưng quan trọng từ dữ liệu đầu vào.

Điểm đột phá của CNN so với các mô hình mạng nơ-ron truyền thống nằm ở khả năng nhận diện các đặc trưng không gian, từ những chi tiết nhỏ như đường viền, góc cạnh, đến các đặc trưng phức tạp hơn như hình dáng vật thể. Điều này được thực hiện thông qua các lớp tích chập (Convolutional Layers) sử dụng các bộ lọc (filters) để quét qua hình ảnh. Quá trình này giúp mô hình không chỉ xác định sự tồn tại của một đặc trưng mà còn cả vị trí của nó, đồng thời làm cho mạng trở nên bất biến trước các dịch chuyển hoặc biến đổi nhỏ trong hình ảnh.

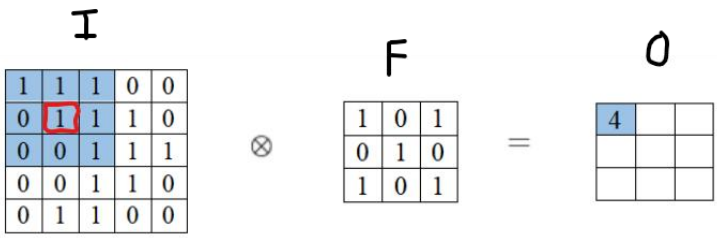
Nhờ vào kiến trúc hiệu quả và khả năng trích xuất đặc trưng vượt trội, CNN đã trở thành nền tảng cho nhiều ứng dụng quan trọng trong lĩnh vực thị giác máy tính như: nhận diện và phân loại hình ảnh (ví dụ: phân biệt mèo và chó), nhận dạng khuôn mặt, phát hiện đối tượng trong thời gian thực, và phân tích hình ảnh y tế (như chẩn đoán khối u từ ảnh X-quang). CNN không chỉ mở ra một kỷ nguyên mới cho các bài toán thị giác máy tính mà còn là một công cụ không thể thiếu trong nghiên cứu và phát triển AI hiện đại.

2.1.1 Lớp tích chập (Convolution layer)

Lớp tích chập sử dụng các bộ lọc filter thực hiện các phép tính tích chập (convolution operations) khi quét đầu vào *I* theo chiều của nó. Các siêu tham số (hyperparameters) bao gồm các kích thước bộ lọc (filter size) *F* và bước nhảy *S*. Đầu ra kết quả *O* được gọi là bản đồ đặc trưng hoặc bản đồ kích hoạt (Feature Map/Activation Map).

2.1.1.1 Phép tính tích chập (convolution operation)

Để dễ hình dung, âm thanh chuyển về dạng 2D spectrogram là một ma trận *I* có kích thước là $H \times W$(Height x Width).



Hình 2. 1 Phép tính tích chập

Giả sử cho ma trận *I* có kích thước là 5*5 và một bộ lọc *F* có kích thước là 3*3. Khi tính tích chập tại vị trí i_{22} như trong hình, với mỗi phần tử của ma trận *I* lấy ra một ma trận X (vùng màu xanh lam) có kích thước bằng với kích thước của bộ lọc. Sau đó tính:

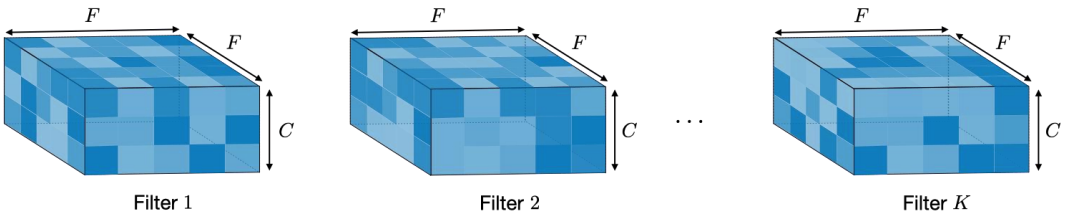
$$y_{11} = sum(\overset{I}{\underset{F}{\bigotimes}}) = i_{11} * f_{11} + i_{12} * f_{12} + i_{13} * f_{13} + i_{21} * f_{21} + i_{22} * f_{22} +$$

$$i_{23} * f_{23} + i_{31} * f_{31} + i_{32} * f_{32} + i_{33} * f_{33} = 4$$

Làm tương tự với các phần tử còn lại trong ma trận.

2.1.1.2 Các siêu tham số của bộ lọc filter (filter hyperparameters)

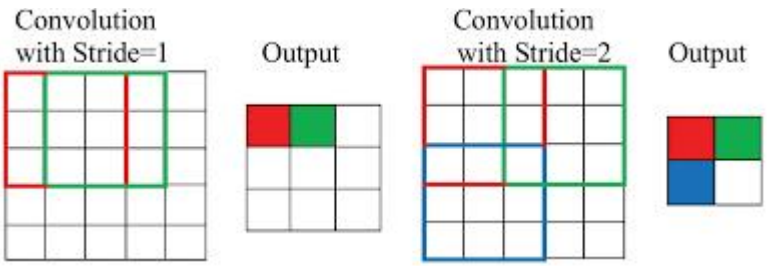
Bộ lọc (Filter) có kích thước $F \times F$ được áp dụng cho một đầu vào chứa c kênh (channels) là một $F \times F \times c$ khối lượng thực hiện phép tính tích chập trên đầu vào có kích thước $i \times i \times c$ và tạo ra một bản đồ đặc trưng đầu ra feature map (còn gọi là bản đồ kích hoạt - activation map) có kích thước $O \times O \times 1$.



Hình 2. 2: K bộ lọc được sử dụng cho ra kết quả một bản đồ đặc trưng có kích thước $O \times O \times K$

Ý nghĩa: trích xuất các đặc trưng từ dữ liệu đầu vào. Mỗi bộ lọc được thiết kế để đọc một đặc trưng cụ thể.

Bước nhảy (Stride) là khoảng cách mà bộ lọc di chuyển sau mỗi lần thực hiện phép tính.



Hình 2. 3: Phép tính tích chập với bước nhảy là 1 và 2

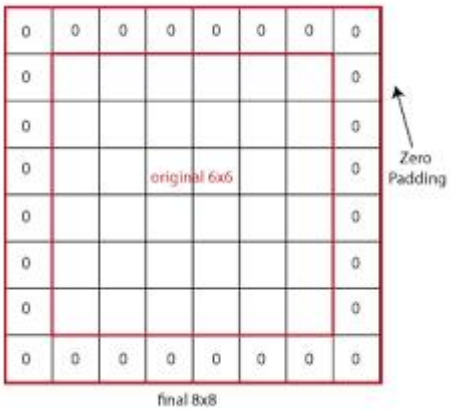
Với bước nhảy là 1 thì bộ lọc nhảy từng ô một.

Với bước nhảy là 2 thì bộ lọc nhảy cách hai ô theo chiều dọc và ngang.

Áp dụng tương tự với bước nhảy lớn hơn.

Ý nghĩa: bước nhảy giúp giảm kích thước của bản đồ đặc trưng, giảm chi phí tính toán và giúp mạng tập trung vào các đặc trưng quan trọng hơn.

Đệm không (Zero padding) là việc thêm P các hàng và cột có giá trị 0 xung quanh viền của dữ liệu đầu vào.

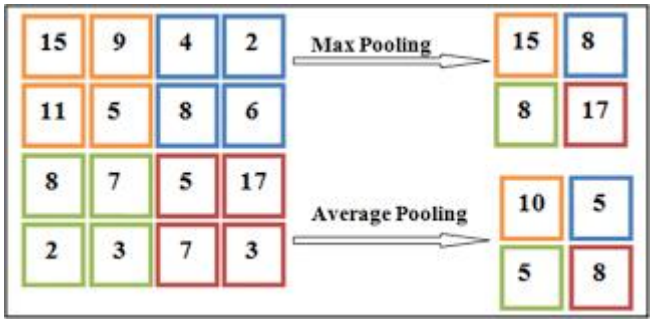


Hình 2. 4: Zero padding

Ý nghĩa: Ngăn chặn việc kích thước của bản đồ đặc trưng bị giảm quá nhanh và đảm bảo rằng các đặc trưng ở rìa của dữ liệu đầu vào cũng được xử lý đầy đủ, vì nếu không có đệm, các đặc trưng này sẽ được quét ít hơn so với các đặc trưng ở trung tâm.

2.1.2 Lớp gộp (Pooling layer)

Lớp gộp là một phép toán giảm mẫu (downsampling operation), thường được áp dụng sau mỗi lớp tích chập, để giảm kích thước dữ liệu nhưng vẫn giữ được các thuộc tính quan trọng, đồng thời cũng giúp giảm các phép tính toán trong mô hình. Có hai loại gộp phổ biến là gộp cực đại (max pooling) và gộp trung bình (average pooling).

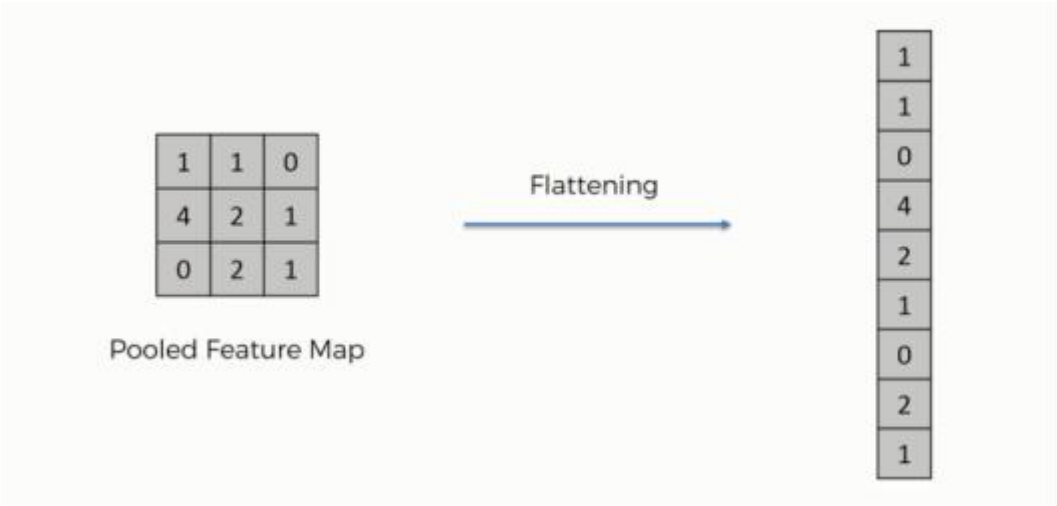


Hình 2. 5: Gộp cực đại và gộp trung bình

Giả sử lớp gộp có kích thước $F \times F$ (2 x 2), ma trận *I* có kích thước (4 x 4) và bước nhảy *S* là 2, đệm *P* là 0. Trên mỗi vùng có kích thước $F \times F$ của *I,* ta tìm phần tử chứa giá trị cực đại hoặc tính trung bình các phần tử rồi viết vào ma trận kết quả *O.*

2.1.3 Lớp kết nối đầy đủ (Fully Connected Layer)

Sau khi ảnh được truyền qua nhiều convolutional layer và pooling layer thì model đã học được tương đối các đặc điểm của ảnh (ví dụ mắt, mũi, khung mặt,…) thì tensor của output của layer cuối cùng, kích thước $H \times W \times K$, sẽ được làm phẳng (flattend) về 1 vector kích thước (H*W*K).



Hình 2. 6: Dữ liệu được làm phẳng khi đưa vào lớp FC

Dựa vào các đặc trưng đã được trích xuất ở các lớp trước, lớp này sẽ đưa ra quyết định cuối cùng để phân biệt giống của chó và mèo là gì.

2.1.4 Điều chỉnh siêu tham số

2.1.4.1 Tương thích tham số trong lớp tích chập (Parameter compatibility in convolution layer)

Các tham số của lớp tích chập phải được chọn sao cho kích thước đầu vào và đầu ra của lớp đó có thể khớp với nhau một cách hợp lệ. Bằng cách ghi chú kích thước *H* (hoặc *W*) của *I,* kích thước *F* của bộ lọc, số lượng đệm P, bước nhảy S, sau đó là kích thước đầu ra *O* của bản đồ đặc trưng dọc theo chiều đó được đưa ra bởi:

$$O = \frac{H(orW) - F + 2P}{S} + 1$$

Tương thích	Không tương thích
I = 7; F = 3; P = 0; S = 2	I = 7; F = 3; P = 0; S = 3
$O = \frac{7-3+2\times 0}{2} + 1$ $= \frac{4}{2} + 1 = 3$	$O = \frac{7-3+2\times 0}{3} + 1$ $= \frac{4}{3} + 1$
Kết quả là một số nguyên, đầu ra là một ma trận 3x3.	Kết quả là không phải là một số nguyên, cho thấy bộ lọc không thể trượt qua đầu vào một cách hiệu quả.

Bảng 2. 1: So sánh tham số tương thích và không tương tích trong tích chập

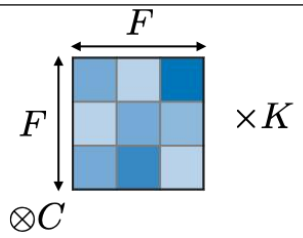
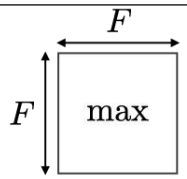
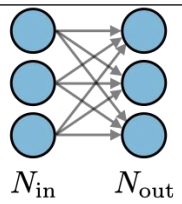
2.1.4.2 Hiểu được độ phức tạp của mô hình (Understanding the complexity of the model)

Hiểu được độ phức tạp của mô hình là rất quan trọng vì nó liên quan trực tiếp đến hai vấn đề then chốt là vị khớp (underfitting) và quá khớp (overfitting).

Vị khớp (Underfitting): nếu mô hình quá đơn giản (độ phức tạp thấp), nó sẽ không đủ khả năng để học được các mẫu cơ bản trong dữ liệu, dẫn đến hiệu suất kém trên cả dữ liệu huấn luyện và dữ liệu kiểm tra.

Quá khớp (Overfitting): nếu mô hình quá phức tạp, nó có thể "học thuộc lòng" dữ liệu huấn luyện, bao gồm cả nhiễu. Điều này dẫn đến hiệu suất rất cao trên dữ liệu huấn luyện nhưng lại rất kém khi gặp dữ liệu mới chưa từng thấy.

Vì vậy, "hiểu được độ phức tạp của mô hình" chính là việc tìm kiếm sự cân bằng lý tưởng: một mô hình đủ phức tạp để học các mẫu cần thiết từ dữ liệu, nhưng không quá phức tạp đến mức bị overfitting. Để đánh giá độ phức tạp của mô hình, việc xác định số lượng tham số mà kiến trúc của nó sẽ rất hữu ích.

	CONV	POOL	FC
Minh họa			
Kích thước đầu vào	$I \times I \times C$	$I \times I \times C$	N_{in}
Kích thước đầu ra	$O \times O \times K$	$O \times O \times C$	N_{out}
Tổng các tham số	$(F \times F \times C + 1) \cdot K$	0	$(N_{in} + 1) \times N_{out}$
Nhận xét	● Một tham số độ lệch (bias) cho mỗi bộ lọc ● Trong hầu hết các trường hợp, $S < F$ ● Một lựa chọn phổ biến để chọn số lượng filter K là gấp đôi số lượng kênh C	● Gộp theo từng kênh ● Trong hầu hết các trường hợp thì $S = F$	● Đầu vào được làm phẳng. ● Một tham số sai lệch (bias) cho mỗi nơ-ron.

Bảng 2. 2: Tổng các tham số trong lớp tích chập và đệm

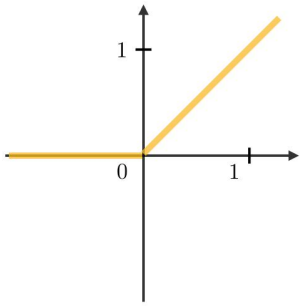
2.1.5 Hàm kích hoạt

Trong một mạng nơ-ron, mỗi nơ-ron sẽ nhận đầu vào từ các nơ-ron khác, tính toán tổng tham số của các đầu vào đó, và sau đó "quyết định" xem nó có nên "kích hoạt" và truyền tín hiệu đó đi tiếp hay không. Hàm kích hoạt (activation function) chính là "người ra quyết định" đó.

2.1.5.1 Hàm kích hoạt ReLU (Rectified Linear Unit)

Hàm kích hoạt ReLU (Rectified Linear Unit) là một trong những hàm kích hoạt phổ biến nhất hiện nay. Đầu ra *O* của lớp tích chập sẽ qua hàm ReLU [3] trước khi trở thành đầu vào *I* của lớp tích chập tiếp theo. Mục đích của nó là đưa các tính chất phi tuyến tính vào mạng. Công thức hàm ReLU:

$$g(z) = \max(0, z)$$



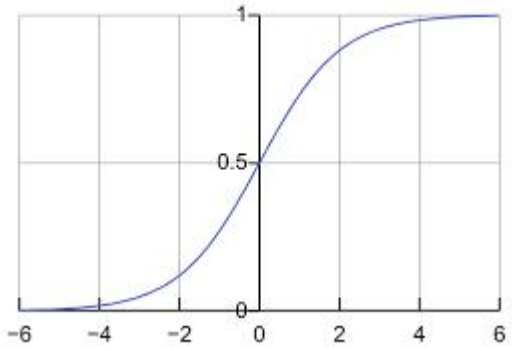
Hình 2. 7: Hàm kích hoạt ReLU

Hàm kích hoạt ReLU cho giá trị đầu ra bằng 0 khi x < 0 và cho ra một hàm tuyến tính với hệ số góc bằng 1 khi x > 0.

2.1.5.2 Hàm kích hoạt Softmax

Softmax là một hàm kích hoạt được sử dụng để chuyển đổi một vector các giá trị thực bất kỳ thành một phân phối xác suất. Điều này có nghĩa là đầu ra của hàm softmax sẽ luôn nằm trong khoảng từ 0 đến 1 và tổng tất cả các giá trị sẽ bằng 1. Với vector đầu vào $z = [z_1, z_2, \ldots, z_n]$, hàm softmax cho phần tử thứ i được tính như sau:

$$softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_i}}$$



Hình 2. 8: Hàm kích hoạt Softmax

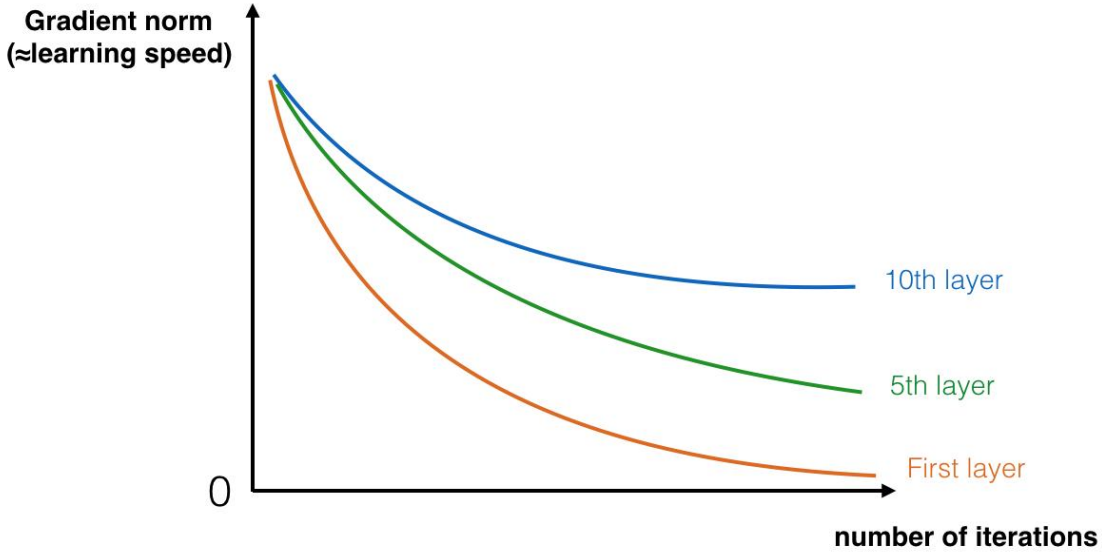
2.2 VẤN ĐỀ VỀ MẠNG NƠ-RON RẤT SÂU

Trong những năm gần đây, mạng nơ-ron đã trở nên sâu hơn, với các mạng tiên tiến từ chỉ vài lớp (ví dụ: AlexNet) đến hơn một trăm lớp. Lợi ích chính của mạng rất sâu là nó có thể biểu diễn các hàm rất phức tạp. Nó cũng có thể học các đặc trưng ở nhiều mức độ trừu tượng khác nhau, từ các cạnh (ở các lớp nông hơn, gần đầu vào hơn) đến các đặc trưng rất phức tạp (ở các lớp sâu hơn, gần đầu ra hơn).

Tuy nhiên, việc sử dụng mạng sâu hơn không phải lúc nào cũng hiệu quả. Một rào cản lớn đối với việc huấn luyện chúng là sự biến mất của gradient: các mạng rất sâu thường có tín hiệu gradient nhanh chóng về 0, do đó làm cho gradient descent chậm đến mức không thể thực hiện được.

Cụ thể hơn, trong quá trình gradient descent, khi backprop từ lớp cuối cùng trở lại lớp đầu tiên, ta đang nhân với ma trận trọng số ở mỗi bước, và do đó gradient có thể giảm nhanh theo cấp số nhân về 0 (hoặc, trong một số trường hợp hiếm hoi, tăng nhanh theo cấp số nhân và "bùng nổ" để lấy các giá trị rất lớn).

Trong quá trình đào tạo, do đó bạn có thể thấy độ lớn (hoặc chuẩn) của độ dốc đối với các lớp nông hơn giảm xuống 0 rất nhanh khi quá trình đào tạo diễn ra:



Hình 2. 9: Sự biến mất của gradient khi mạng rất sâu

2.3 RESNET (RESIDUAL NETWORK)

Mạng phần dư (residual network), là một loại mạng nơ-ron được thiết kế để giải quyết vấn đề của các mạng nơ-ron truyền thống nhất là khi mạng trở nên quá sâu (quá nhiều lớp).

Trong ResNets, "lối tắt" (shortcut) hoặc "kết nối bỏ qua" (skip connection) cho phép mô hình bỏ qua các lớp:

Hình 2. 10: Mối khối ResNet hiện thị "skip connection"

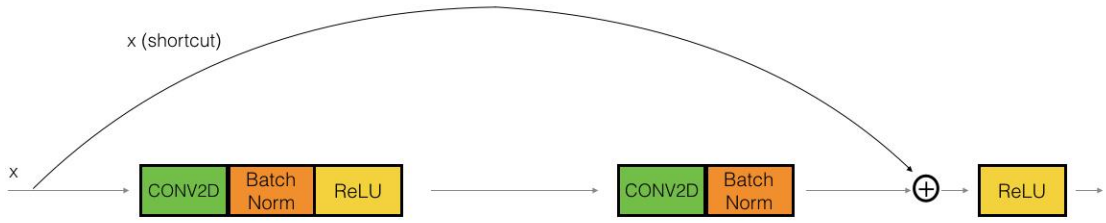
Hình ảnh bên trái cho thấy "đường dẫn chính" xuyên qua mạng. Hình ảnh bên phải thêm một lối tắt vào đường dẫn chính. Bằng cách xếp chồng các khối ResNet này lên nhau, bạn có thể tạo thành một mạng rất sâu.

Cũng có một số bằng chứng cho thấy việc dễ dàng học một hàm đồng nhất là yếu tố quyết định hiệu suất đáng chú ý của ResNet [8], thậm chí còn hơn cả việc bỏ qua các kết nối giúp giảm gradient.

Có hai khối chính được sử dụng trong ResNet là "khối đồng nhất" (identity block) và "khối tích chập" (convolution block).

2.3.1 Khối đồng nhất (The identity block)

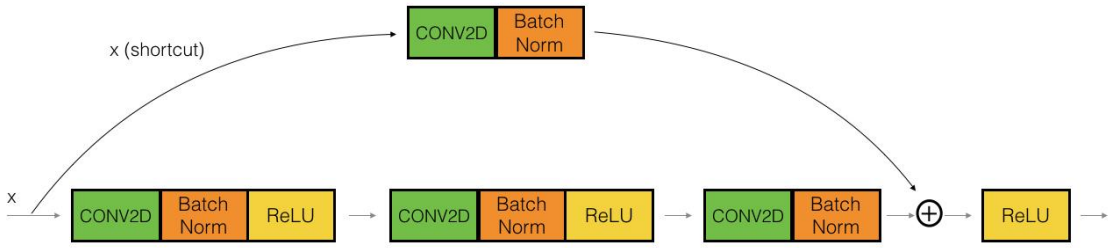
Khối định danh là khối tiêu chuẩn được sử dụng trong ResNet và tương ứng với trường hợp kích hoạt đầu vào ($a^{[l]}$) có cùng kích thước với kích hoạt đầu ra ($a^{[l+2]}$). Để làm rõ các bước khác nhau diễn ra trong khối định danh của ResNet, sau đây là sơ đồ thay thế hiển thị từng bước riêng lẻ:



Hình 2. 11: Khối đồng nhất với kết nối bỏ qua 2 lớp

2.3.2 Khối tích chập (The convolutional block)

"Khối tích chập" ResNet là loại khối thứ hai. Điểm khác biệt với khối định danh là có một lớp CONV2D trong đường dẫn tắt:



Hình 2. 12: Khối tích chập trong ResNet

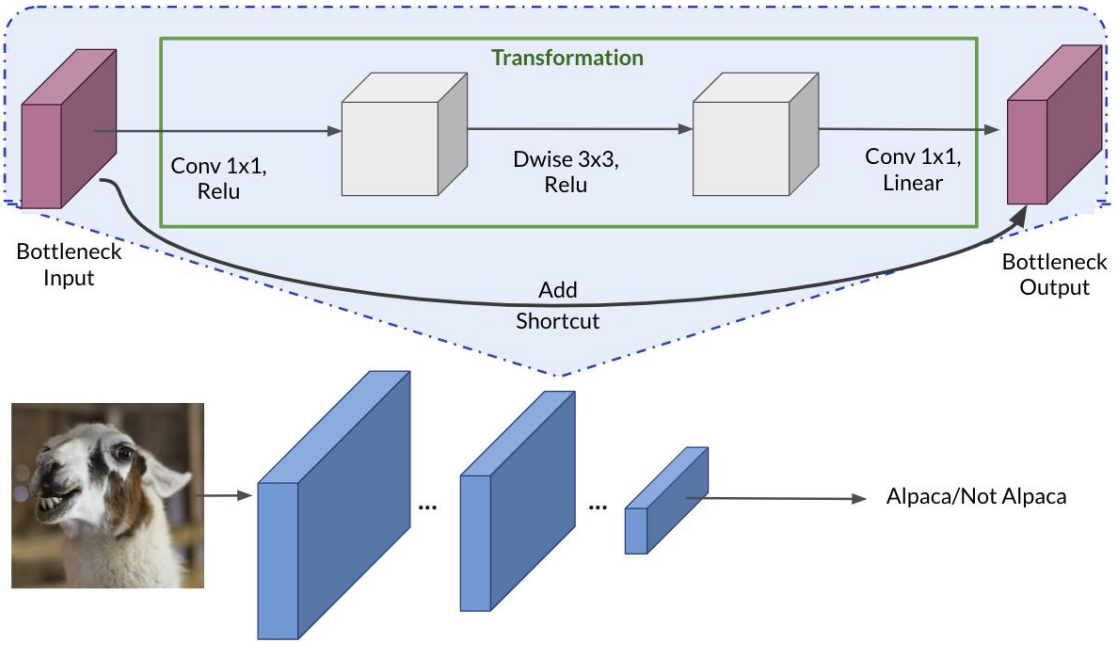
Lớp CONV2D trong đường dẫn tắt được sử dụng để thay đổi kích thước đầu vào thành một chiều khác, sao cho các chiều khớp nhau trong phép cộng cuối cùng cần thiết để thêm giá trị lối tắt trở lại đường dẫn chính. Ví dụ: để giảm chiều

cao và chiều rộng của các chiều kích hoạt xuống 2 lần, ta có thể sử dụng tích chập 1x1 với bước nhảy là 2.

Lớp CONV2D trên đường dẫn tắt không sử dụng bất kỳ hàm kích hoạt phi tuyến tính nào. Vai trò chính của nó chỉ là áp dụng một hàm tuyến tính (đã học) để giảm chiều của đầu vào, sao cho các chiều khớp nhau cho bước cộng sau đó.

2.4 MOBILENET_V2

MobileNetV2 [10] là một kiến trúc mạng nơ-ron tích chập (CNN) được thiết kế đặc biệt để tối ưu hóa hiệu suất và hiệu quả trên các thiết bị di động, nhúng có tài nguyên hạn chế. MobileNetV2 sử dụng tích chập phân tách theo chiều sâu (depth-wise separable convolution) làm nền tảng hiệu quả.



Hình 2. 13: Kiến trúc MobileNet_V2

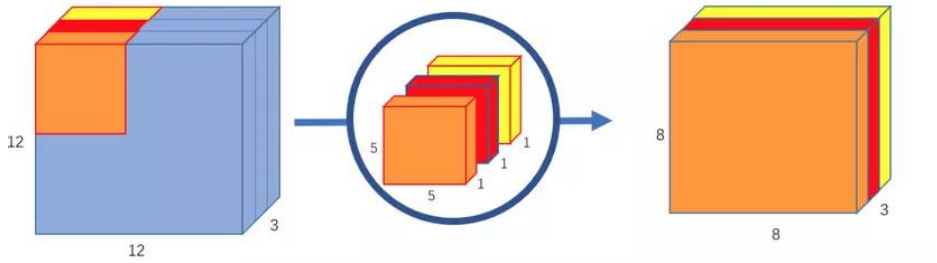
Mỗi khối bao gồm một cấu trúc dư ngược với một nút thắt ở mỗi đầu. Các nút thắt này mã hóa các đầu vào và đầu ra trung gian trong một không gian chiều thấp, và ngăn chặn các tính phi tuyến tính phá hủy thông tin quan trọng. Các kết nối tắt, tương tự như các kết nối trong các mạng dư truyền thống, phục vụ cùng một mục đích là tăng tốc quá trình huấn luyện và cải thiện dự đoán. Các kết nối này bỏ qua các phép tích chập trung gian và kết nối các lớp nút thắt.

2.4.1 Tích chập phân tách theo chiều sâu (depth-wise separable convolution)

Tích chập phân tách theo chiều sâu (depth-wise separable convolution) [4] được thiết kế để giảm đáng kể số lượng tham số và chi phí tính toán so với tích chập thông thường, giúp mô hình hoạt động hiệu quả hơn trên các thiết bị có tài nguyên hạn chế. Kỹ thuật này chia quá trình tích chập ra thành hai giai đoạn riêng biệt là tích chập theo chiều sâu (depth-wise convolution) và tích chập điểm (point-wise convolution).

2.4.1.1 Tích chập theo chiều sâu (Depth-wise Convolution)

Đầu tiên ở giai đoạn này, thực hiện tích chập trên input *I* có kích thước 12x12x3 mà không làm thay đổi số lượng kênh *C* của ảnh, bằng cách sử dụng 3 bộ lọc *F* có kích thước 5x5x1 với không đệm *P* và bước nhảy *S* là 1.



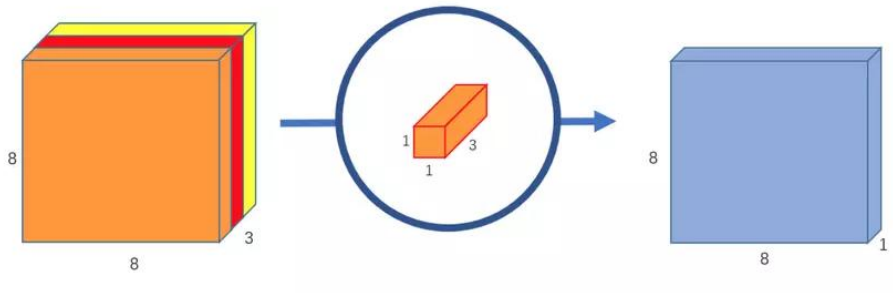
Hình 2. 14: Tích chập theo chiều sâu, sử dụng 3 bộ lọc để thay đổi kích thước đầu vào 12x12x3 sang 8x8x3

Ở đây, mỗi bộ lọc sẽ thực hiện phép tích chập với mỗi kênh *C* của *I* cho ra đầu ra có kích thước 8x8x1, gộp các kết quả lại, ta thu được đầu ra *O* kích thước 8x8x3.

2.4.1.2 Tích chập điểm (Point-wise Convolution)

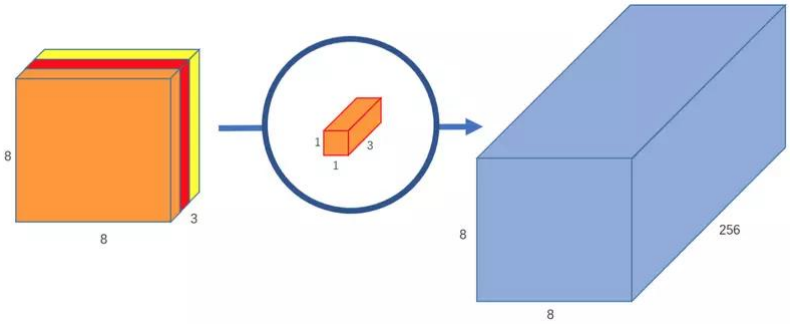
Sau khi giai đoạn tích chập theo chiều sâu hoàn thành, kết quả sẽ được xử lý bởi một lớp tích chập khác. Mục đích kết hợp thông tin từ các kênh *C* khác nhau và tạo ra một bản đồ đặc trưng mới *O* có số lượng kênh mong muốn.

Giai đoạn này sẽ sử dụng các filter 1x1xC để thực hiện phép tích chập với từng điểm dữ liệu. Filter sẽ có số lượng kênh bằng với số lượng kênh của ảnh đầu vào với mục đích để thu được 1 kênh của ảnh đầu ra.



Hình 2. 15: Tích chập điểm, chuyển đổi đầu vào có 3 kênh sang 1 kênh

Ở đây, filter của chúng ta sẽ là 1x1x3, sau khi convolution thu được đầu ra 8x8x1. Tiếp theo là tạo ra số kênh C bằng với số kênh đầu vào, giả dụ số kênh đầu vào là 256.



Hình 2. 16: Tích chập điểm với 256 kênh

2.4.1.3 So sánh với tích chập thông thường

Với phép tích chập thông thường, thực hiện tính tích chập với 256 bộ lọc 5x5x3 trên 8x8 điểm dữ liệu đầu vào *I,* tức là 256x5x5x3x8x8 = 1,228,800 phép nhân.

Với tích chập phân tách theo chiều sâu, ở giai đoạn depthwise convolution, chúng ta có 3 bộ lọc 5x5x1 dịch chuyển 8x8 lần, nghĩa là 3x5x5x1x8x8=4,800 phép nhân. Còn pointwise convolution, chúng ta có 256 bộ lọc 1x1x3 dịch chuyển 8x8 lần, 256x1x1x3x8x8=49,152 phép nhân, cộng 2 cái lại chúng ta mất 53,952 phép nhân tất cả.

Chúng ta đều thấy được rằng 53,952 nhỏ hơn 1,228,800 rất nhiều, mô hình mạng của chúng ta sẽ nhẹ hơn và có khả năng thực hiện nhanh hơn rất nhiều so với phép tích chập thông thường.

2.5 EFFICIENTNETV2_S

Năm 2019 nhóm nghiên cứu của Google đã phát hành bài báo liên quan tới họ CNN mới - EfficientNet [12]. Mô hình CNN này không làm tăng accuracy mà còn cải thiện hiệu suất của model bằng cách giảm số parameters và FLOPS (Floating Point Operations Per Second) so với các mô hình hiện đại như GPipe. Những đóng góp chính của bài báo này bao gồm:

+ Thiết kế simple mobile-size baseline architecture (kiến trúc cơ sở đơn giản kích thước di động): EfficientNet-B0

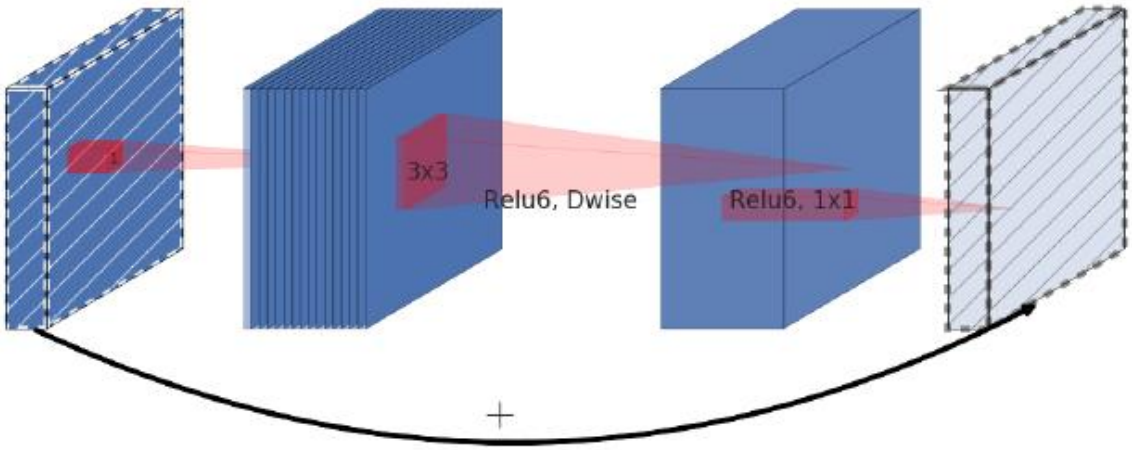
+ Cung cấp phương pháp compound scaling (mở rộng kết hợp) một cách hiệu quả giúp tăng kích thước model để đạt được độ chính xác cao nhất.

Stage i	Operator $\hat{\mathcal{F}}_i$	Resolution $\hat{H}_i \times \hat{W}_i$	#Channels $\hat{C}_i$	#Layers $\hat{L}_i$
1	Conv3x3	224×224	32	1
2	MBConv1, k3x3	112×112	16	1
3	MBConv6, k3x3	112×112	24	2
4	MBConv6, k5x5	56×56	40	2
5	MBConv6, k3x3	28×28	80	3
6	MBConv6, k5x5	14×14	112	3
7	MBConv6, k5x5	14×14	192	4
8	MBConv6, k3x3	7×7	320	1
9	Conv1x1 & Pooling & FC	7×7	1280	1

Bảng 2.3: Các lớp thuộc kiến trúc mạng EfficientNet-B0

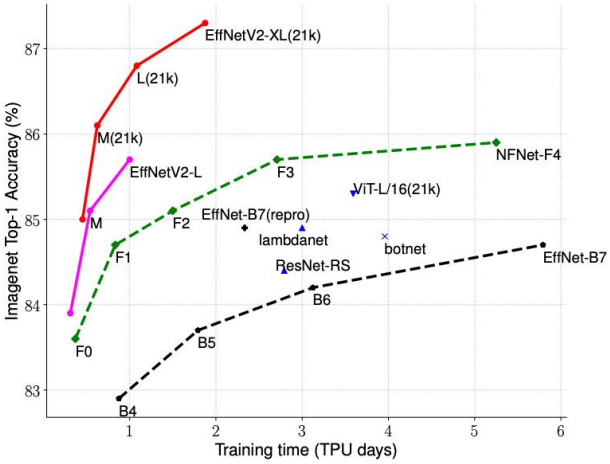
Building block chính của EfficientNet-B0 là MBConv block. MBConv block tương tự như inverted residual block được sử dụng trong MobileNet v2. Trong block này có shortcut connection (kết nối tắt) giữa phần đầu và cuối của block. Phần input được mở rộng bằng Conv layer 1x1 để tăng số channels hay depth của feature map. Sau đó chúng ta sử dụng Depthwise convolution 3x3 và Pointwise convolution (Conv layer 1x1) để giảm số channels của output. Shortcut connection kết nối các narrow layers (layers hẹp - số channels nhỏ) trong khi các

wider layers (layers rộng hơn) nằm ở giữa shortcut connection. Cấu trúc này giúp giảm số lượng parameters và số lượng operations tương ứng.



Hình 2. 17: Khối dư đảo ngược (Inverted residual block)

EfficientNetV2 là một họ các mô hình phân loại hình ảnh, đạt hiệu quả tham số tốt hơn và tốc độ huấn luyện nhanh hơn so với các công nghệ trước đây. Được xây dựng dựa trên EfficientNetV1, các mô hình EfficientNetV2 sử dụng tìm kiếm kiến trúc nơ-ron (NAS) để tối ưu hóa đồng thời kích thước mô hình và tốc độ huấn luyện, đồng thời được mở rộng quy mô để tăng tốc độ huấn luyện và suy luận.



(a) Training efficiency.

	EfficientNet (2019)	ResNet-RS (2021)	DeiT/ViT (2021)	EfficientNetV2 (ours)
Top-1 Acc.	84.3%	84.0%	83.1%	83.9%
Parameters	43M	164M	86M	24M

(b) Parameter efficiency.

Hình 2. 18: Kết quả khi so sánh độ chính xác và các tham số của kiến trúc EfficientNetV2 với một số kiến trúc khác

Mô hình Efficientnetv2_S [11] với chữ 'S' trong tên gọi đại diện cho "Small" (nhỏ). Dòng EfficientNetV2 bao gồm các phiên bản khác nhau về kích thước như EfficientNetV2-S, EfficientNetV2-M (Medium), EfficientNetV2-L (Large) và các phiên bản lớn hơn nữa. Mỗi phiên bản được tạo ra bằng cách áp dụng một quy tắc mở rộng (scaling rule) cụ thể để điều chỉnh độ sâu, chiều rộng và độ phân giải, phù hợp với các nhu cầu tài nguyên khác nhau.

2.6 TRANSFER LEARNING

2.6.1.1 Học chuyển giao (transfer learning) là gì?

Trong các mô hình ResNet, Efficientnet và MobileNet sau khi được xử lý và tính toán, mô hình sẽ học được các đặc điểm của ảnh, trước khi đưa vào lớp FC. ResNet, Efficientnet và MobileNet trong ImageNet model [9] cũng lấy ra các đặc điểm của hình dạng chó và mèo (tai, mũi, mắt,...).

Quá trình sử dụng lại các mô hình như ImageNet được đào tạo trước được gọi là pre-trained model. Mô hình được đào tạo trước là một mạng đã được đào tạo trên một tập dữ liệu lớn và được lưu lại, cho phép ta sử dụng nó để tùy chỉnh mô hình của riêng mình một cách tiết kiệm và hiệu quả.

ImageNet là một trong những bộ dữ liệu lớn nhất và phổ biến nhất trong lĩnh vực thị giác máy tính, chứa hơn 14 triệu hình ảnh thuộc hơn 1000 thể loại khác nhau.

Có 2 loại transfer learning như sau:

Feature extractor: Sau khi lấy ra các đặc điểm của ảnh bằng việc sử dụng ConvNet của pre-trained model, thì ta sẽ dùng linear classifier (linear SVM, softmax classifier,..) để phân loại ảnh. Hiểu đơn giản thì các đặc điểm ảnh (tai, mũi, tóc,…) giờ như input của bài toán linear regression hay logistic regression.

Fine tuning: Sau khi lấy ra các đặc điểm của ảnh bằng việc sử dụng pre-trained model, ta sẽ tinh chỉnh, bổ sung thêm các lớp để nhận diện thêm các đặc điểm trên bộ dữ liệu mà ta muốn xử lý, giống như việc mượn một cuốn bách khoa toàn thư đã được viết sẵn về thế giới động vật, sau đó chỉ cần thêm một vài trang chuyên sâu về các giống chó và mèo.

2.6.1.2 Khi nào thì nên dùng tranfer learning

Có 2 yếu tố quyết định để chúng ta sử dụng tranfer learning là kích thước dữ liệu mà ta cần xử lý và sự tương đồng của dữ liệu giữa mô hình ta cần train và pre-trained model.

Dữ liệu ta có nhỏ và tương tự với dữ liệu ở pre-trained model. Vì dữ liệu nhỏ nên nếu dùng fine-tuning thì model sẽ bị overfitting.

Dữ liệu ta có lớn và tương tự với dữ liệu ở pre-trained model. Giờ có nhiều dữ liệu ta không sợ overfitting do đó nên dùng fine-tuning.

Dữ liệu bạn có lớn và tương tự với dữ liệu ở pre-trained model. Giờ có nhiều dữ liệu ta không sợ overfitting do đó nên dùng fine-tuning.

Dữ liệu bạn có lớn và khác với dữ liệu ở pre-trained model. Ta có thể train model từ đầu, tuy nhiên sẽ tốt hơn nếu ta khởi tạo các giá trị weight của model với giá trị của pre-trained model và sau đó train bình thường.

CHƯƠNG 3

MÔ HÌNH THỰC NGHIỆM

3.1 THU THẬP VÀ TIỀN XỬ LÝ DỮ LIỆU

3.1.1 Tổng quan về dữ liệu

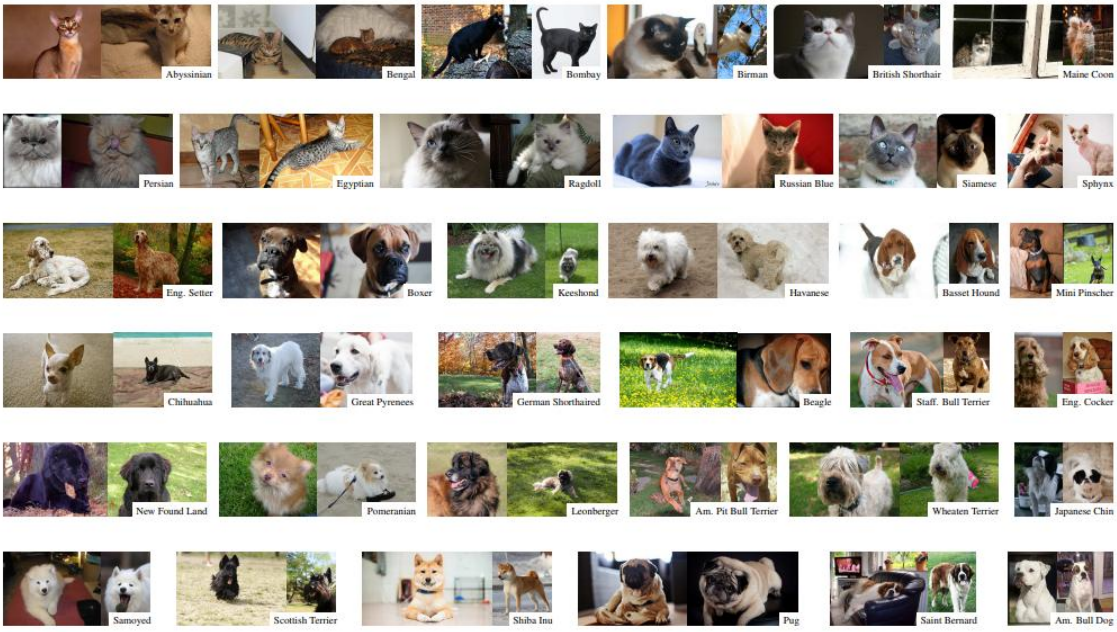
Nghiên cứu này sử dụng bộ dữ liệu Oxford-IIIT Pet [13]. Bộ dữ liệu được tạo và chia sẻ bởi đại học Oxford và viện công nghệ thông tin quốc tế tại Ấn Độ.

Oxford-IIIT Pet là một tập hợp các hình ảnh đã được chú thích (annotated) của mèo và chó, được tạo ra để phục vụ cho các nghiên cứu về phân loại đối tượng chi tiết (fine-grained object categorization). Vấn đề chính mà bộ dữ liệu này giải quyết là xác định giống chó hoặc mèo từ một hình ảnh.

Bộ dữ liệu chứa 7.390 hình ảnh của 37 giống mèo và chó khác nhau. Cụ thể, có 12 giống mèo và 25 giống chó. Mỗi giống có khoảng 200 hình ảnh.

```
Abyssinian                    : 200
Bengal                        : 200
Birman                        : 200
Bombay                        : 200
British_Shorthair             : 200
Egyptian_Mau                  : 200
Maine_Coon                    : 200
Persian                       : 200
Ragdoll                       : 200
Russian_Blue                  : 200
Siamese                       : 200
Sphynx                        : 200
american_bulldog              : 200
american_pit_bull_terrier     : 200
basset_hound                  : 200
beagle                        : 200
boxer                         : 200
chihuahua                     : 200
english_cocker_spaniel        : 200
english_setter                : 200
german_shorthaired            : 200
great_pyrenees                : 200
havanese                      : 200
japanese_chin                 : 200
keeshond                      : 200
leonberger                    : 200
miniature_pinscher            : 200
newfoundland                  : 200
pomeranian                    : 200
pug                           : 200
saint_bernard                 : 200
samoyed                       : 200
scottish_terrier              : 199
shiba_inu                     : 200
staffordshire_bull_terrier    : 191
wheaten_terrier               : 200
yorkshire_terrier             : 200
```

Hình 3. 1: Thành phần dữ liệu Oxford-IIIT. 12 giống mèo và 25 giống chó



Hình 3. 2: Một số hình ảnh từ bộ dữ liệu Oxford-IIIT Pet

Các tác giả đã đề xuất một mô hình để tự động phân loại giống thú cưng từ hình ảnh, kết hợp cả đặc điểm hình dạng (shape) và đặc điểm ngoại hình của bộ lông (appearance). Các mô hình này đã đạt được kết quả rất đáng khích lệ, với độ chính xác trung bình khoảng 59% khi phân biệt 37 giống khác nhau, cho thấy đây là một vấn đề đầy thách thức.

3.1.2 Tải và xử lý dữ liệu

3.1.2.1 Tải dữ liệu

Bộ dữ liệu được tải thông qua lệnh wget từ hai đường dẫn URL bao gồm:

Tập tin thứ nhất chứa hình ảnh:

http://www.robots.ox.ac.uk/~vgg/data/pets/data/images.tar.gz

Tập tin thứ hai chứa thông tin chú thích (nhãn):

http://www.robots.ox.ac.uk/~vgg/data/pets/data/annotations.tar.gz

Hai tập tin sau khi tải sẽ được giải nén và đưa vào list comprehension để tạo danh sách các đường dẫn ảnh một cách ngắn gọn và hiệu quả.

```
# Tạo danh sách đường dẫn đến tập ảnh và tập nhãn
input_dir = "images/"
target_dir = "annotations/trimaps/"

# Sắp xếp ảnh jpg và nhãn của nó.
input_img_paths = sorted(
    [os.path.join(input_dir, fname)
     for fname in os.listdir(input_dir)
     if fname.endswith(".jpg")])
target_paths = sorted(
    [os.path.join(target_dir, fname)
     for fname in os.listdir(target_dir)
     if fname.endswith(".png") and not fname.startswith(".")])
```

Hình 3. 3: Tạo đường dẫn chứa thông tin đã được sắp xếp

Đường dẫn ảnh (input_img_paths) và đường dẫn nhãn (target_paths).

```
plt.axis("off")
plt.imshow(load_img(input_img_paths[5000]));
```

Hình 3. 4: Hiển thị ảnh 5001 trong input_img_paths

3.1.2.2 Tiền xử lý dữ liệu và nhãn

Phần này chuyển đổi dữ liệu thô (ảnh và đường dẫn) thành định dạng phù hợp cho mô hình.

Trích xuất nhãn: xử lý chuỗi cơ bản từ đường dẫn để lấy tên giống.

```
# Tạo danh sách mới chứa tên nhãn
target = []

# Loại bỏ đường dẫn '/' và đuôi đặc trưng của tệp
# annotations/trimaps/Abyssinian_1.png thành Abyssinian
for breed in target_paths:
  target.append(breed.split('/',2)[2].rsplit('_',1)[0])
```

Hình 3. 5: Lấy nhãn từ đường dẫn

Mã hóa nhãn (labelEncDict): đưa các nhãn về dạng số nguyên để mô hình học.

```python
# Nhận danh sách tên nhãn và trả về một mảng Numpy
list_breeds  = np.unique(target)

# Loài mèo với chữ ký tự đầu viết hoa, chó thì viết thường
cats_vs_dogs = [0 if i[0].isupper() == True else 1 for i in target]

# Tạo từ điển labelEncDict, gán mỗi nhãn duy nhất một chỉ số số nguyên.
labelEncDict = {name : ind for ind, name in enumerate(list_breeds)}

# Chuẩn hóa nhãn (Label encoding) và hiển thị tên gốc và nhãn đã được chuẩn hóa
for k, v in labelEncDict.items():
    print(f"{k:32} : {v}")
```

Hình 3. 6: Mã hóa nhãn khi huấn luyện (labelEncDict)

Giải mã nhãn (labelDecDict): dùng để giải mã kết quả dự đoán trở lại tên giống loài thực tế, giúp kết quả dễ hiểu hơn.

```python
# Giải mã nhãn (Label Decoding)
labelDecDict = {ind: name for name, ind in labelEncDict.items()}
for k, v in labelDecDict.items():
    print(f"{k:3} : {v}")
```

```
  0 : Abyssinian
  1 : Bengal
  2 : Birman
  3 : Bombay
  4 : British_Shorthair
  5 : Egyptian_Mau
  6 : Maine_Coon
  7 : Persian
  8 : Ragdoll
  9 : Russian_Blue
 10 : Siamese
 11 : Sphynx
 12 : american_bulldog
 13 : american_pit_bull_terrier
 14 : basset_hound
 15 : beagle
 16 : boxer
```

Hình 3. 7: Giải mã nhãn (labelDecDict) và hiển thị kết quả khi giải mã

Tạo mảng và xử lý ảnh: path_to_input_image(): Hàm này chuyển đổi ảnh sang mảng NumPy, và sử dụng tf.image.resize_with_pad để chuẩn hóa kích thước tất cả các ảnh về 224x224 pixels. Điều này rất cần thiết vì các mô hình học sâu yêu cầu tất cả đầu vào có cùng kích thước.

```python
# Hàm xử lý hình ảnh
def path_to_input_image(path):
    img = load_img(path)
    # thay đổi kích thước ảnh và thêm padding
    img = tf.image.resize_with_pad(img_to_array(img, dtype = 'float32'), *img_size).numpy().astype('uint8')
    return img

# Tạo danh sách rỗng input_imgs
input_imgs = []
targets = np.array(target)

# Duyệt và gọi hàm, thêm kết quả vào danh sách input_imgs
for i in range(num_imgs):
    input_imgs.append(path_to_input_image(input_img_paths[i]))

# Chuyển đổi các mảng hình ảnh thành một mảng Numpy duy nhất
input_imgs = np.array(input_imgs)
```

Hình 3. 8: Chuyển ảnh về mảng numpy và đưa về kích thước 224x224

Hình 3. 9: Trực quan hóa vài dữ liệu sau khi đã tiền xử lý

3.1.3 Phân chia và chuẩn hóa dữ liệu

3.1.3.1 Phân chia dữ liệu cho huấn luyện mô hình

Sử dụng hàm train_test_split() từ thư viện scikit-learn để chia dữ liệu.

Việc chia dữ liệu làm hai bước (X_tv, X_test, ... và sau đó X_train, X_val, ...) là một kỹ thuật tốt để đảm bảo mô hình được đánh giá trên một tập dữ liệu kiểm tra hoàn toàn độc lập, chưa bao giờ nhìn thấy trong quá trình huấn luyện.

Tham số stratify=y_tv (và stratify=targets) là rất quan trọng. Nó đảm bảo rằng tỷ lệ các lớp (giống loài) trong mỗi tập dữ liệu con (huấn luyện, xác thực, kiểm tra) là tương đương với tỷ lệ trong tập dữ liệu ban đầu. Điều này giúp ngăn ngừa việc một số giống loài bị thiếu trong tập kiểm tra, dẫn đến kết quả đánh giá không chính xác.

```
IMAGE_SIZE = (224, 224)
RANDOM_STATE = 42
TRAIN_SIZE, VAL_SIZE, TEST_SIZE = 0.8, 0.1, 0.1

X_tv, X_test, y_tv, y_test = train_test_split(
    input_imgs,
    labelsEncoded,
    test_size = TEST_SIZE,
    random_state = RANDOM_STATE,
    stratify = targets
    )

X_train, X_val, y_train, y_val = train_test_split(
    X_tv,
    y_tv,
    test_size = VAL_SIZE,
    random_state = RANDOM_STATE,
    stratify = y_tv
    )
```

Hình 3. 10: Chia dữ liệu với thư viện thư viện scikit-learn

```
Training Data: (5985, 224, 224, 3)
Training Labels: 5985

Validation Data: (666, 224, 224, 3)
Validation Labels: 666

Testing Data: (739, 224, 224, 3)
Testing Labels: 739
```

Hình 3. 11: Kết quả sau khi chia dữ liệu huấn luyện

3.1.3.2 Chuẩn hóa dữ liệu

Chuẩn hóa dữ liệu: rescale=1./255 đưa các giá trị pixel từ khoảng [0, 255] về [0, 1]. Hầu hết các mô hình học sâu hoạt động tốt nhất với dữ liệu đầu vào trong khoảng này.

```
train_gen = ImageDataGenerator(rescale = 1./255,
                               #rotation_range = 30,
                               #width_shift_range = 0.1,
                               #height_shift_range = 0.1,
                               #shear_range = 0.1,
                               #zoom_range = 0.1,
                               #horizontal_flip = True,
                               fill_mode = 'nearest',
                               #preprocessing_function=to_grayscale_then_rgb)
                               )
train_data = train_gen.flow(x = X_train, y = y_train, batch_size = 32, shuffle = True)
len(train_data)
```

```
188
```

```
val_gen = ImageDataGenerator(rescale = 1./255, )

val_data = val_gen.flow(x = X_val, y = y_val, batch_size = 32, shuffle = True)
len(val_data)
```

```
21
```

```
test_gen = ImageDataGenerator(rescale = 1./255, )

test_data = test_gen.flow(x = X_test, y = y_test, batch_size = 32, shuffle=False) # False
len(test_data)
```

```
24
```

Hình 3. 12: Chuẩn hóa dữ liệu về khoảng [0, 1]

3.2 XÂY DỰNG VÀ HUẤN LUYỆN MÔ HÌNH

Quá trình xây dựng và huấn luyện mô hình được thực hiện dựa trên phương pháp học chuyển giao (transfer learning). Đây là một kỹ thuật mạnh mẽ trong học sâu, cho phép tận dụng các mô hình đã được huấn luyện sẵn trên các tập dữ liệu lớn để giải quyết một bài toán mới với lượng dữ liệu hạn chế. Cụ thể, mô hình sẽ sử dụng các trọng số (weights) đã học được từ tập dữ liệu ImageNet, một tập dữ liệu lớn gồm hàng triệu hình ảnh, và tinh chỉnh chúng để phù hợp với bài toán phân loại giống chó mèo.

3.2.1 Kiến trúc mô hình hoàn chỉnh

Mô hình được xây dựng theo kiến trúc Sequential của thư viện Keras. Kiến trúc này bao gồm một chuỗi các lớp (layer) được xếp chồng lên nhau, từ đầu vào đến đầu ra. Mô hình sử dụng các mô hình nền tảng (backbone) đã được huấn luyện sẵn, được nhập từ thư viện TensorFlow Hub. Ba kiến trúc nền tảng được thử nghiệm bao gồm MobileNetV2, ResNet50, EfficientNetV2-S.

Lớp thứ nhất: hub.KerasLayer(pretrained_model, trainable=False). Đây là lớp quan trọng nhất, nhập mô hình nền tảng đã được huấn luyện. Tham số trainable=False có nghĩa là các trọng số của mô hình này sẽ được "đóng băng" và không thay đổi trong quá trình huấn luyện. Điều này cho phép tận dụng các tính năng học được của mô hình gốc, trong khi chỉ huấn luyện các lớp mới được thêm vào.

Lớp thứ hai: tf.keras.layers.Flatten(). Lớp này chuyển đổi ma trận đầu ra hai chiều của mô hình nền tảng thành một vector một chiều, chuẩn bị dữ liệu để đưa vào các lớp kết nối đầy đủ (Dense layers) tiếp theo.

Lớp thứ ba và thứ năm: tf.keras.layers.Dense(..., activation='relu'). Đây là các lớp kết nối đầy đủ với 128 và 64 nơ-ron tương ứng, sử dụng hàm kích hoạt ReLU (rectified linear unit). Các lớp này chịu trách nhiệm học các đặc trưng cấp cao, đặc thù cho bài toán phân loại giống chó mèo.

Lớp thứ tư và thứ sáu: tf.keras.layers.Dropout(0.3). Lớp này được thêm vào để giảm hiện tượng quá khớp (overfitting). Trong quá trình huấn luyện, nó sẽ ngẫu nhiên "tắt" 30% số nơ-ron của lớp trước đó, buộc mô hình phải học các biểu diễn mạnh mẽ hơn từ các nơ-ron còn lại.

Lớp cuối cùng: tf.keras.layers.Dense(37, activation='softmax'). Đây là lớp đầu ra với 37 nơ-ron, tương ứng với 37 giống loài trong tập dữ liệu. Hàm kích hoạt softmax chuyển đổi đầu ra của lớp này thành một phân phối xác suất, giúp xác định xác suất của mỗi giống loài cho một bức ảnh đầu vào.

```
pretrained_base = hub.KerasLayer(classifier_model[name], input_shape=IMAGE_SIZE + (3,), trainable = False)
model = tf_keras.Sequential([
    pretrained_base,
    tf_keras.layers.Flatten(),
    tf_keras.layers.Dense(128, activation='relu'),
    tf_keras.layers.Dropout(0.3),
    tf_keras.layers.Dense(64, activation='relu'),
    tf_keras.layers.Dropout(0.3),
    tf_keras.layers.Dense(37, activation='softmax'),
])
```

Hình 3. 13: Kiến trúc mô hình hoàn chỉnh

3.2.2 Cấu Hình và Quá Trình Huấn Luyện

Trước khi bắt đầu huấn luyện, mô hình được biên dịch (compile) với các tham số sau:

Bộ tối ưu hóa (Optimizer): 'adam'. Đây là một thuật toán tối ưu hóa phổ biến và hiệu quả, giúp mô hình điều chỉnh trọng số một cách nhanh chóng để giảm thiểu hàm mất mát.

Hàm mất mát (Loss function): 'sparse_categorical_crossentropy'. Hàm này rất phù hợp cho các bài toán phân loại đa lớp khi nhãn đầu ra được biểu diễn dưới dạng các số nguyên.

Đo lường (Metrics): ['accuracy']. Độ chính xác được sử dụng làm thước đo chính để đánh giá hiệu suất của mô hình trong suốt quá trình huấn luyện.

```
model.compile(
    optimizer='adam',
    loss = 'sparse_categorical_crossentropy',
    metrics=['accuracy'],
)
```

Hình 3. 14: Biên dịch mô hình

```
history = model.fit(
    train_data,
    validation_data= val_data,
    epochs=6,
    steps_per_epoch=train_steps_per_epoch,
    validation_steps=val_steps_per_epoch,
)
histories[name] = history
```

Hình 3. 15: Huấn luyện mô hình với 6 epochs

Mô hình được huấn luyện trong 6 epoch (epochs=6). Dữ liệu huấn luyện và xác thực được cung cấp dưới dạng các batch thông qua các đối tượng ImageDataGenerator đã được cấu hình trước. Quá trình huấn luyện này được lặp lại cho mỗi kiến trúc nền tảng (MobileNetV2, ResNet50, EfficientNetV2-S), và kết quả (history) được lưu lại để so sánh hiệu suất giữa các mô hình.

3.3 KẾT QUẢ

Quá trình huấn luyện được thực hiện trong 6 epoch cho mỗi mô hình. Các chỉ số chính được theo dõi là hàm mất mát (loss) và độ chính xác (accuracy) trên cả tập dữ liệu huấn luyện và tập dữ liệu xác thực.

Model	**Train Acc**	**Val Acc**
Mobilenet_v2	0.8901	0.8769
Efficientnetv2_s	0.9069	0.8784
Resnet50	0.8520	0.8574

Bảng 3.1: Độ chính xác (Acc) trên tập huấn luyện (train) và tập xác thực (val)

Model	**Train Loss**	**Val Loss**
Mobilenet_v2	0.3406	0.4871
Efficientnetv2_s	0.2812	0.4059
Resnet50	0.4556	0.5702

Bảng 3.2: Độ mất mát (Loss) trên tập huấn luyện (train) và tập xác thực (val)

EfficientNetV2-S thể hiện hiệu suất vượt trội nhất trong số ba mô hình được thử nghiệm. Mô hình này không chỉ đạt độ chính xác huấn luyện cao nhất (90.69%) mà còn có độ chính xác xác thực cao nhất (87.84%), chứng tỏ khả năng học và tổng quát hóa tốt nhất.

MobileNetV2 có độ chính xác huấn luyện và xác thực thấp hơn một chút, sự chênh lệch là không đáng kể. Điều này khẳng định MobileNetV2 là một lựa chọn hiệu quả, đặc biệt khi cân nhắc về tốc độ và tài nguyên tính toán.

So với hai mô hình còn lại, ResNet50 có hiệu suất thấp nhất trên cả hai tập dữ liệu. Độ chính xác huấn luyện (85.20%) và xác thực (85.74%) đều thấp hơn đáng kể. Kết quả này cho thấy ResNet50 không phải là lựa chọn tối ưu cho tập dữ liệu và cấu hình huấn luyện đã sử dụng.

CHƯƠNG 4

KẾT LUẬN VÀ KIẾN NGHỊ

4.1 KẾT LUẬN

Trong nghiên cứu này, tôi đã xây dựng và đánh giá hiệu suất của ba mô hình học sâu, sử dụng kỹ thuật học chuyển giao (transfer learning) để giải quyết bài toán nhận dạng giống chó mèo từ tập dữ liệu Oxford-IIIT Pet Dataset. Ba kiến trúc CNN nền tảng được thử nghiệm là MobileNetV2, ResNet50 và EfficientNetV2-S.

Dựa trên kết quả thực nghiệm, mô hình EfficientNetV2-S đã chứng tỏ hiệu quả vượt trội, đạt độ chính xác xác thực cao nhất là 87.84%. MobileNetV2 cũng là một lựa chọn rất mạnh mẽ với độ chính xác 87.69%, chỉ kém 0.15% so với mô hình dẫn đầu. Ngược lại, ResNet50 cho hiệu suất thấp nhất trong cả ba mô hình, đạt độ chính xác 85.74%.

Kết quả này khẳng định rằng học chuyển giao là một phương pháp hiệu quả để xây dựng mô hình phân loại hình ảnh, ngay cả với tập dữ liệu tương đối nhỏ. Việc sử dụng các mô hình đã được huấn luyện sẵn giúp giảm đáng kể thời gian và tài nguyên tính toán cần thiết, đồng thời đạt được độ chính xác cao.

4.2 KHIẾN NGHỊ

Để cải thiện hiệu suất của mô hình và khắc phục một số hạn chế của đồ án này, tôi đề xuất các hướng phát triển sau:

1. Cải thiện quá trình tiền xử lý và tăng cường dữ liệu: Mặc dù đã sử dụng ImageDataGenerator, việc áp dụng các kỹ thuật tăng cường dữ liệu phong phú hơn như xoay ảnh, lật ngang, điều chỉnh độ sáng và độ tương phản có thể giúp mô hình mạnh mẽ hơn và ít bị quá khớp.

2. Huấn luyện các lớp nền tảng (Fine-Tuning): Thay vì đóng băng hoàn toàn các trọng số của mô hình nền tảng (trainable=False), chúng tôi có thể thử huấn luyện một phần hoặc toàn bộ các lớp của mô hình đó với một tốc độ học rất nhỏ.

Điều này cho phép mô hình điều chỉnh các đặc trưng đã học để phù hợp hơn với dữ liệu cụ thể của bài toán.

3. Thử nghiệm các kiến trúc và siêu tham số khác: Mặc dù đã thử ba mô hình, vẫn còn nhiều kiến trúc học sâu khác có thể được đánh giá. Ngoài ra, việc điều chỉnh các siêu tham số (hyperparameters) như tốc độ học, số lượng nơ-ron trong các lớp Dense, hoặc tỷ lệ Dropout cũng có thể dẫn đến kết quả tốt hơn.

Sử dụng tập dữ liệu lớn hơn: Mặc dù dữ liệu đã dùng là khá phổ biến, việc mở rộng sang các tập dữ liệu lớn hơn và đa dạng hơn có thể giúp mô hình học được các đặc trưng phong phú hơn và đạt hiệu suất tổng quát hóa cao hơn nữa.

Tài liệu tham khảo

[1]. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324, 1998.

[2]. Ronneberger, O., Fischer, P., Brox, T.: U-net: Convolutional networks for biomedical image segmentation. In: Medical Image Computing and Computer-Assisted Intervention (MICCAI), LNCS, vol. 9351, pp. 234–241, 2015.

[3]. Agarap, A.F.M.: Deep Learning using Rectified Linear Units (ReLU). arXiv preprint arXiv:1803.08375, 2018.

[4]. A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv preprint arXiv:1704.04861, 2017.

[5]. V. Dumoulin and F. Visin, "A Guide to Convolution Arithmetic for Deep Learning," arXiv preprint arXiv:1603.07285, 2016.

[6]. O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in Med. Image Comput. Comput.-Assist. Intervent.–MICCAI, Munich, Germany, 2015, pp. 234–241.

[7]. C. Guo, M. Szemenyei, Y. Yi, W. Wang, B. Chen, and C. Fan, "SA-UNet: Spatial Attention U-Net for Retinal Vessel Segmentation," arXiv preprint arXiv:2004.03696, 2020.

[8]. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770-778, 2016.

[9]. J. Deng, W. Dong, R. Socher, L. Li, K. Li, and F. Li, "ImageNet: A large-scale hierarchical image database," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 248–255, 2009.

[10]. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in Proceedings of

the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4510-4520, 2018.

[11]. M. Tan and Q. Le, "EfficientNetV2: Smaller Models and Faster Training," in Proceedings of the 38th International Conference on Machine Learning, pp. 10096-10106, 2021.

[12]. M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proceedings of the 36th International Conference on Machine Learning, pp. 6105–6114, 2019.

[13]. O. M. Parkhi, A. Vedaldi, A. Zisserman and C. V. Jawahar, "Cats and Dogs," 2011 International Conference on Computer Vision, Barcelona, Spain, 2011, pp. 349-356.